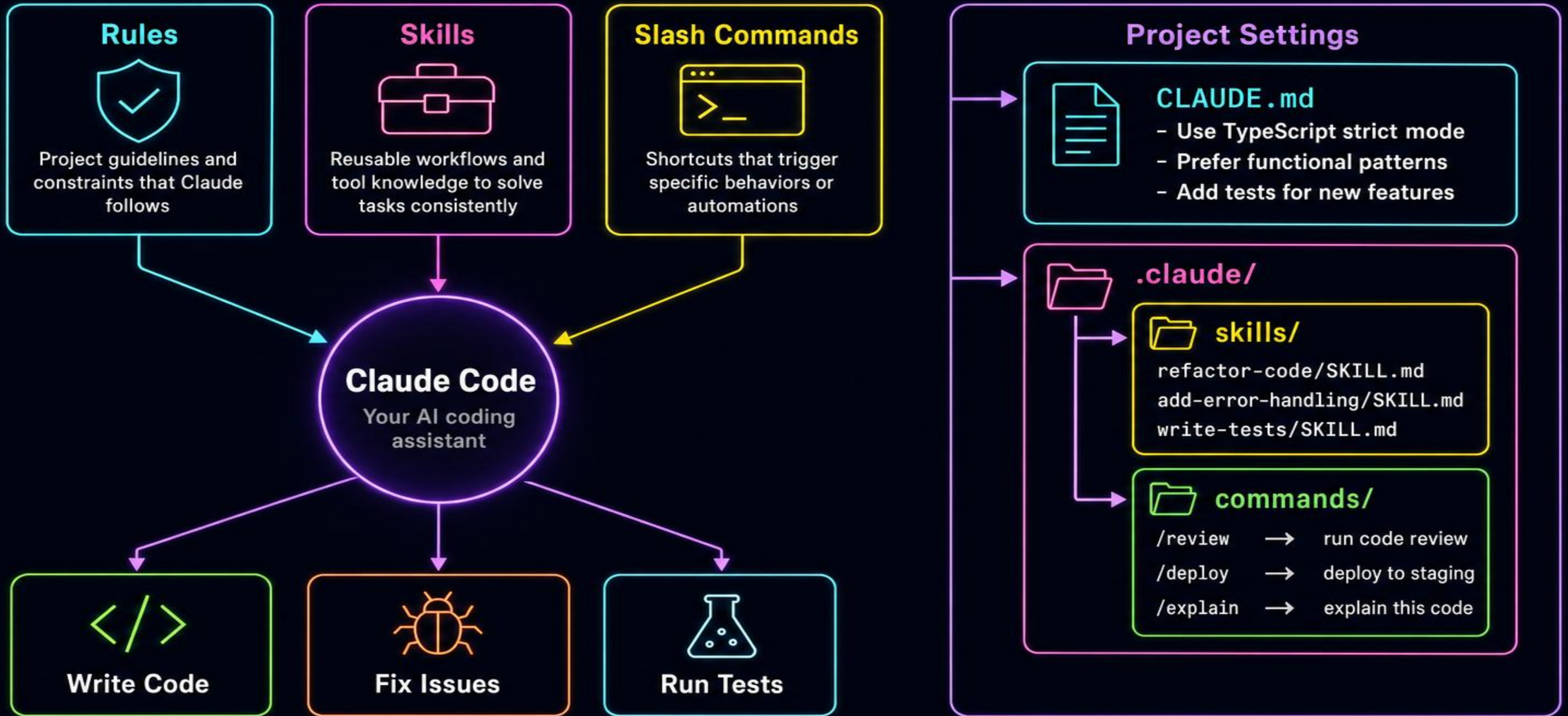


Rules, Skills, and Slash Commands

Claude Code gives you several ways to shape how it works in your project. This lesson covers three of them.

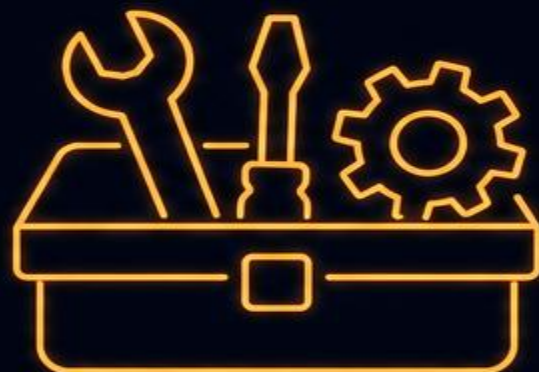


Three Ways to Steer Claude Code



Rules

Standing, always-on instructions in `CLAUDE.md` that load into context at session start and apply to every turn without being re-invoked.



Skills

Reusable capabilities packaged as a `SKILL.md` file with YAML frontmatter that Claude loads only when the current task matches its description.



Slash Commands

Slash-prefixed shortcuts you type in a session. Built-in ones like `/init` ship with the CLI. Custom ones are Markdown files you author.

Rules vs. Skills



Rules (always on)

Loaded into context at launch and applied on every turn. You never invoke them with a slash. They persuade Claude, they don't enforce, so weigh them as guidance.



Skills (on demand)

Only the description sits in context so Claude knows the skill exists. The body loads when you or the model invoke it, which keeps long procedures cheap until needed.

Rules Load at Launch, Not on Demand.

Rules behave the opposite of skills, and mixing them up is the classic trap.



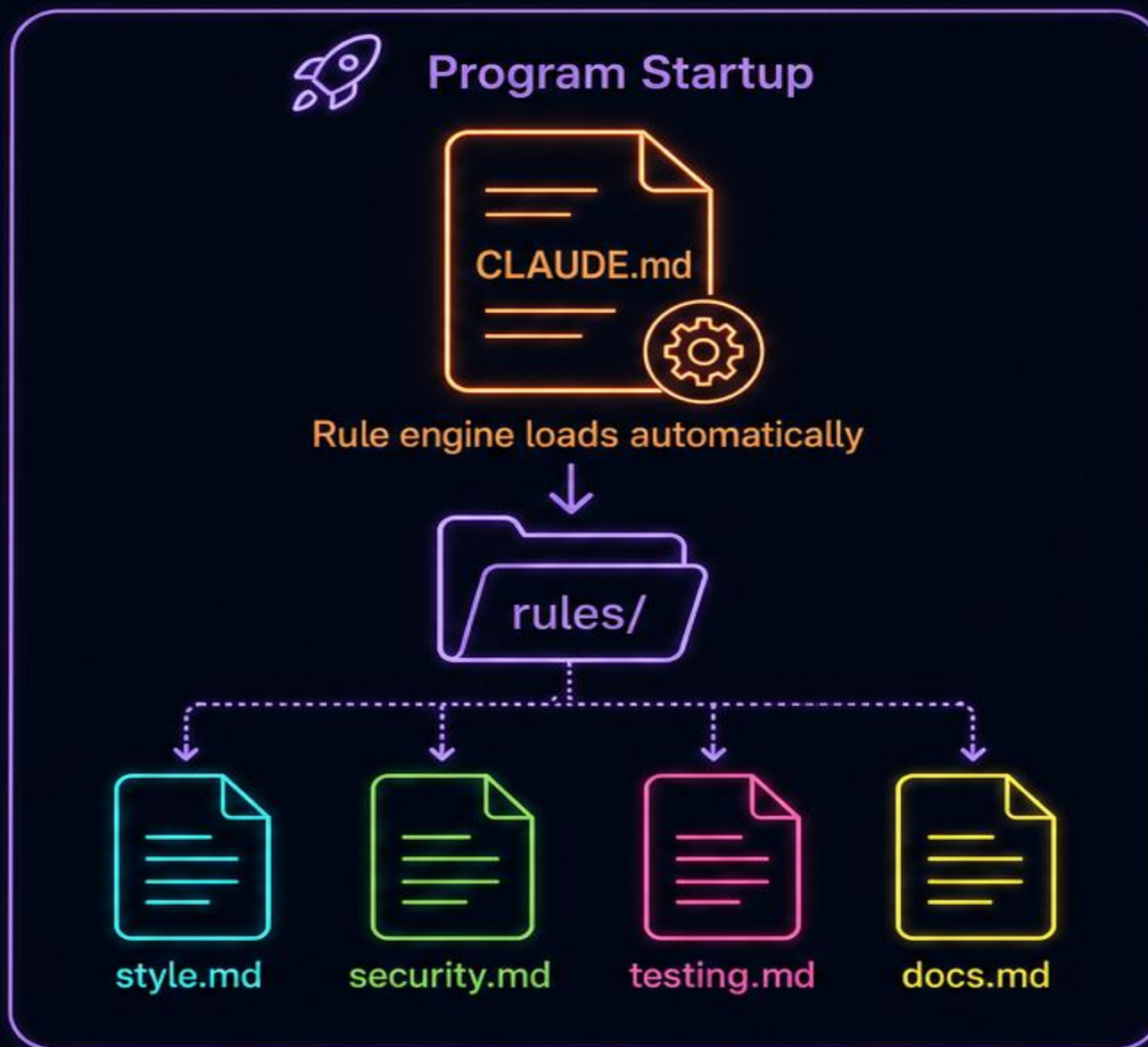
They load at session start,
or when you touch files
their paths match



You never trigger a rule
with a slash command



Split from CLAUDE.md
into topic files under
a rules folder



What a Skill Is Made Of

A skill is a small folder Claude can pick up and use on demand.



web_search



SKILL.md

```
1 ---
2 name: web-search
3 description: Search the web and return
4   relevant results with sources.
5 ---
6
7 # Web Search Skill
8
9 Use the web_search tool to find current
10 information from the web and summarize
11 the most relevant results.
```



A **SKILL.md entry file** holds the instructions



YAML frontmatter carries the description Claude matches on



The folder name becomes what you type after the slash

Only the description is truly needed for the model to find it.

Why a Bare File Fires Badly



Frontmatter recommended

Frontmatter fields are optional. Name defaults to the folder, description to the first paragraph, but a bare file matches poorly.



Description is the trigger

The model matches on the description to decide relevance. A weak or missing one makes the skill fire unreliably.



Placement still matters

The file has to sit in a skills or commands folder Claude scans. A note elsewhere stays just a note.

Commands and Skills Are the Same Thing

Older tutorials treat slash commands and skills as two separate features. In current Claude Code, they were merged.



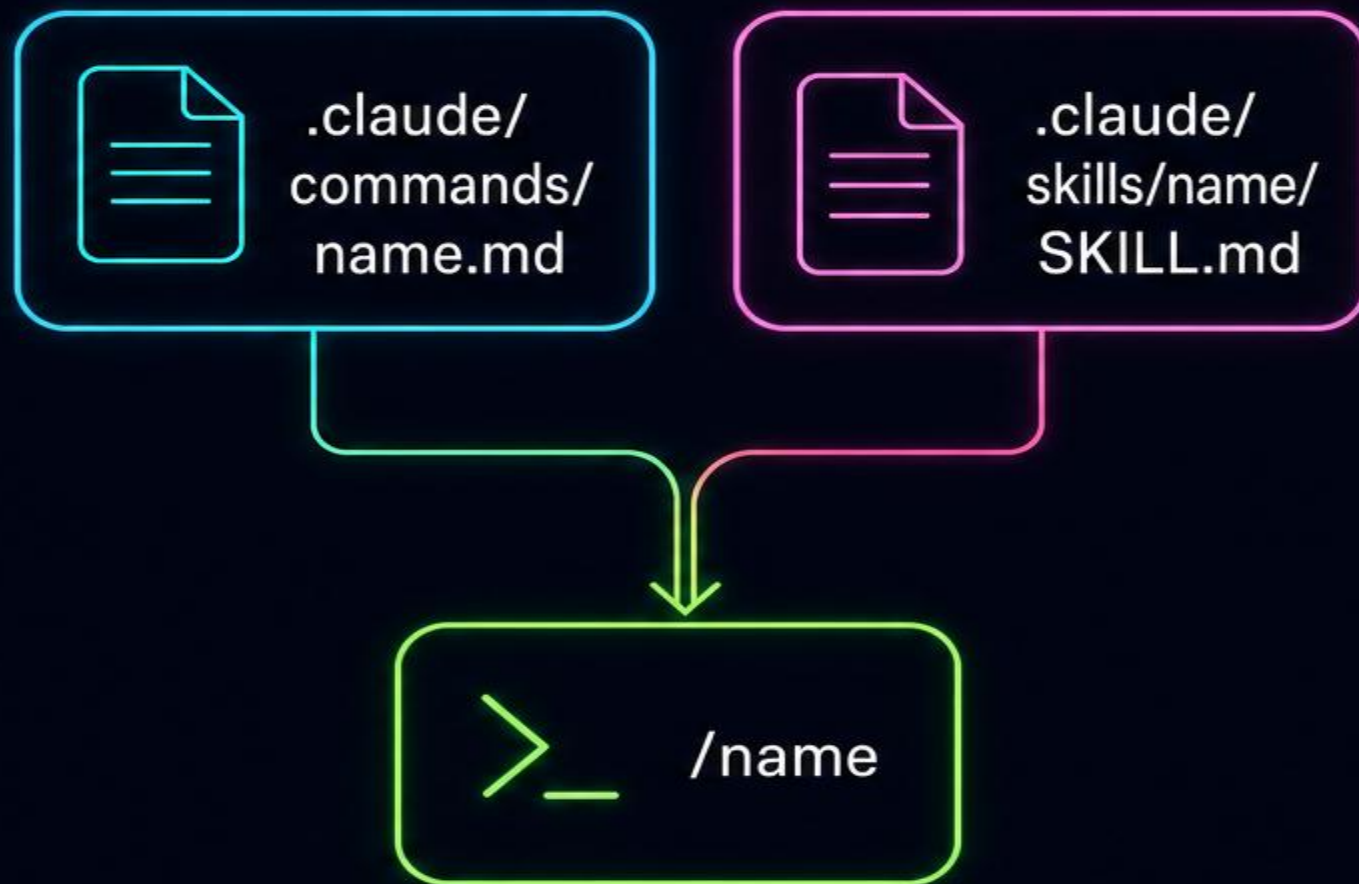
A file at `.claude/commands/name.md`
creates `/name`



A skill at `.claude/skills/name/SKILL.md`
also creates `/name`



Both run the same way,
so a custom command is a skill



When a skill and command share a name, the skill wins.

Built-in vs. Custom Commands



Built-in commands

Fixed logic coded into the CLI, like `/init`, `/clear`, `/help`, and `/model`. They run deterministic behavior you can't edit, and they ship with the tool.

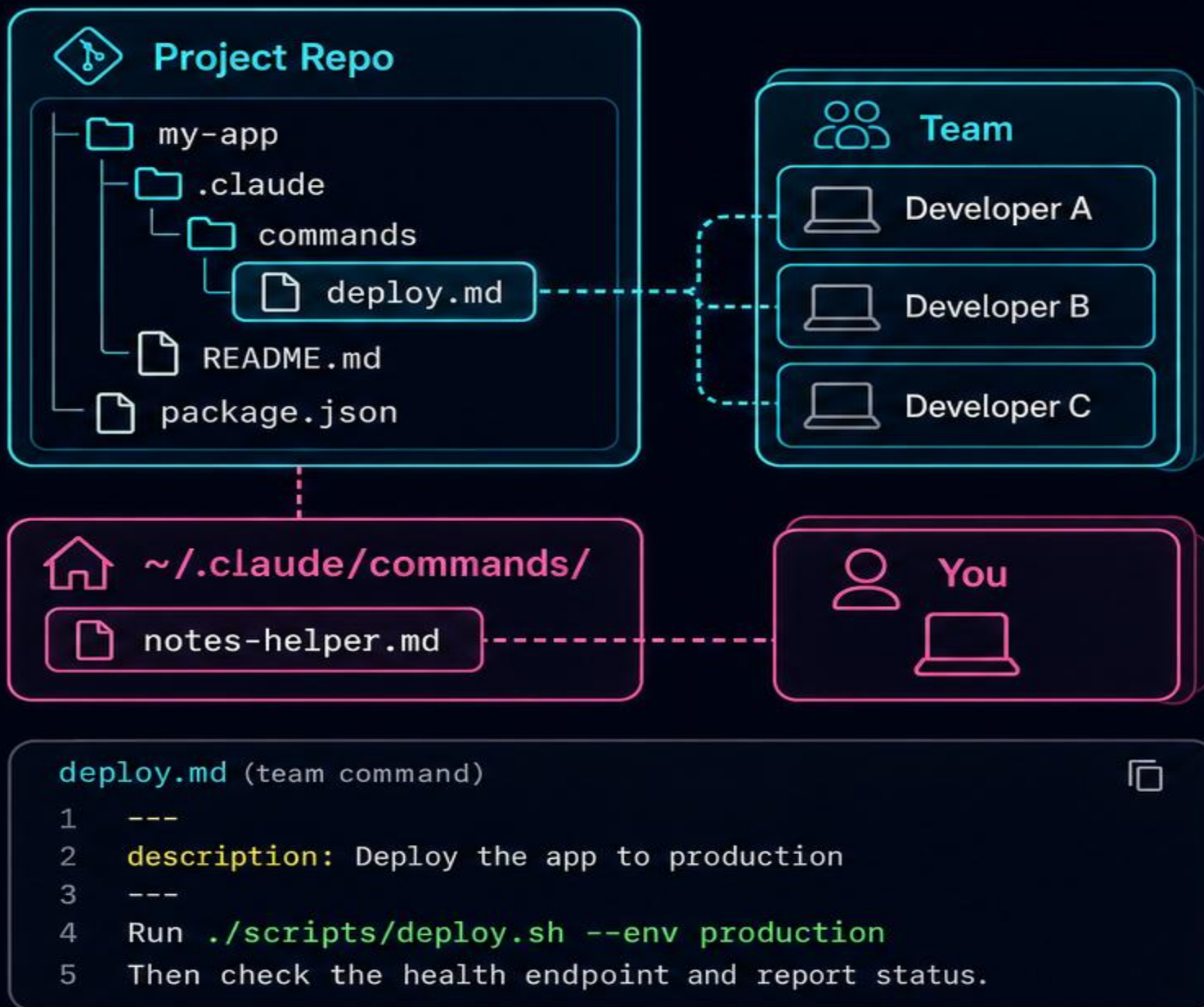


Custom commands

Markdown files you write yourself. The filename becomes the slash keyword, and you control exactly what the command tells Claude to do.

Location Sets the Scope

Where you put a custom command decides who gets it, with no registration step.



 **.claude/commands/** in the repo shares it with the whole team

 **~/.claude/** keeps it personal across all your projects

 **Check the project version** in, and everyone gets the same command

Steering Claude Code



Rules load automatically and apply to every turn



Skills load only when a task matches their description



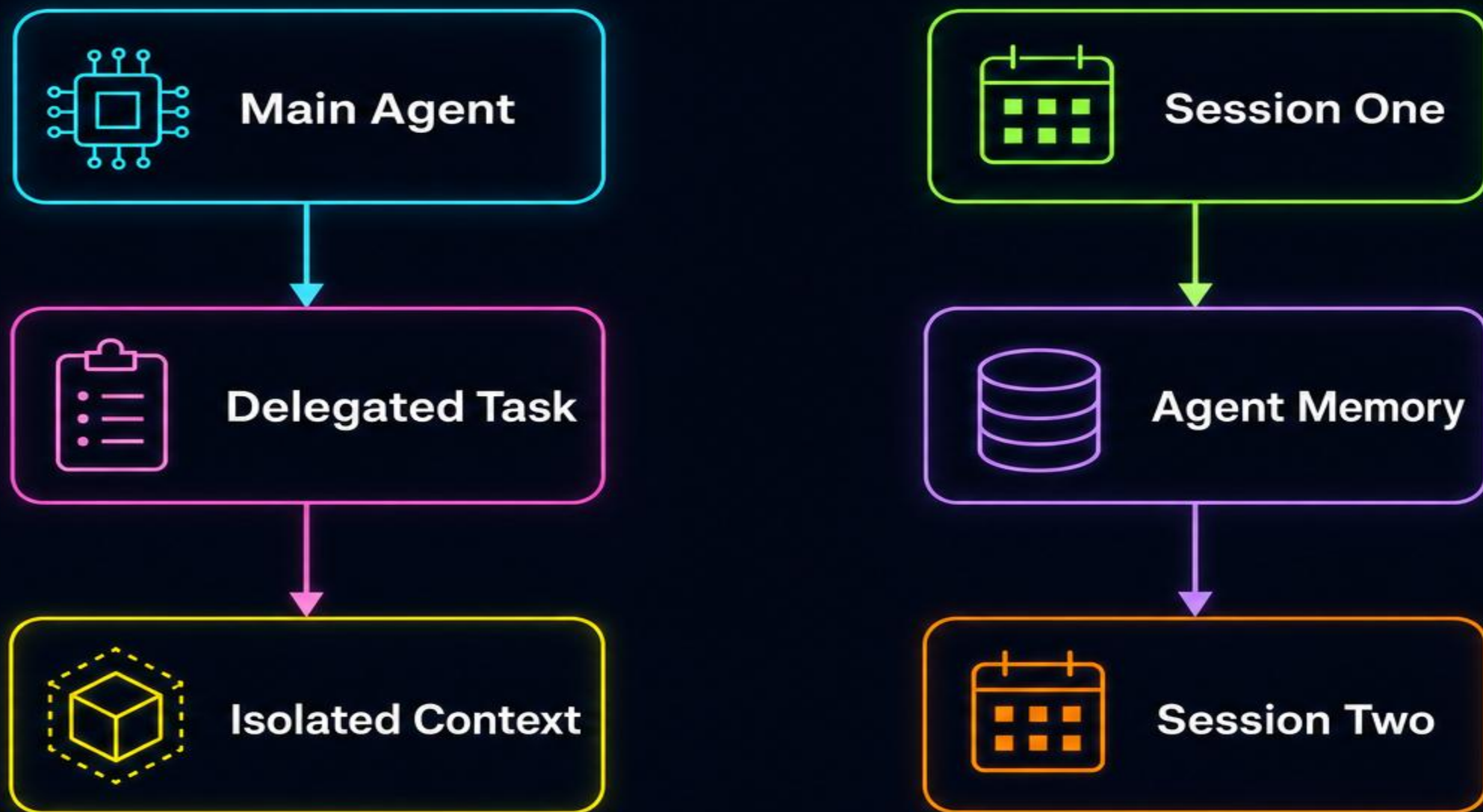
A bare file with no clear description fires badly, so always write one



Custom commands are skills, and file location sets their scope

Subagents and Agent Memory

A subagent is a separate Claude Code instance that handles a delegated task in its own isolated context.



Three Terms for Delegation



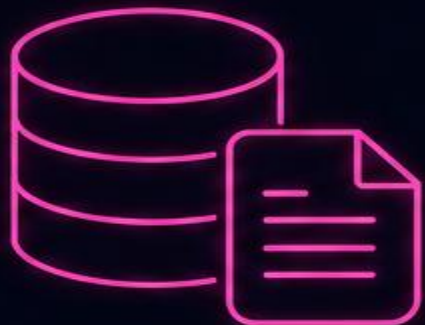
Subagent

A separate Claude Code instance spawned to handle a delegated subtask in an isolated context, with its own tools and permissions.



Isolated Context

The subagent's own context window. It doesn't see your conversation history, and only a summary returns to the main agent.



Agent Memory

Persistent state an agent keeps across sessions, outside the live context window. It survives compaction and session boundaries.

What Makes a Subagent Different

The defining trait of a subagent is a fresh, separate context window.



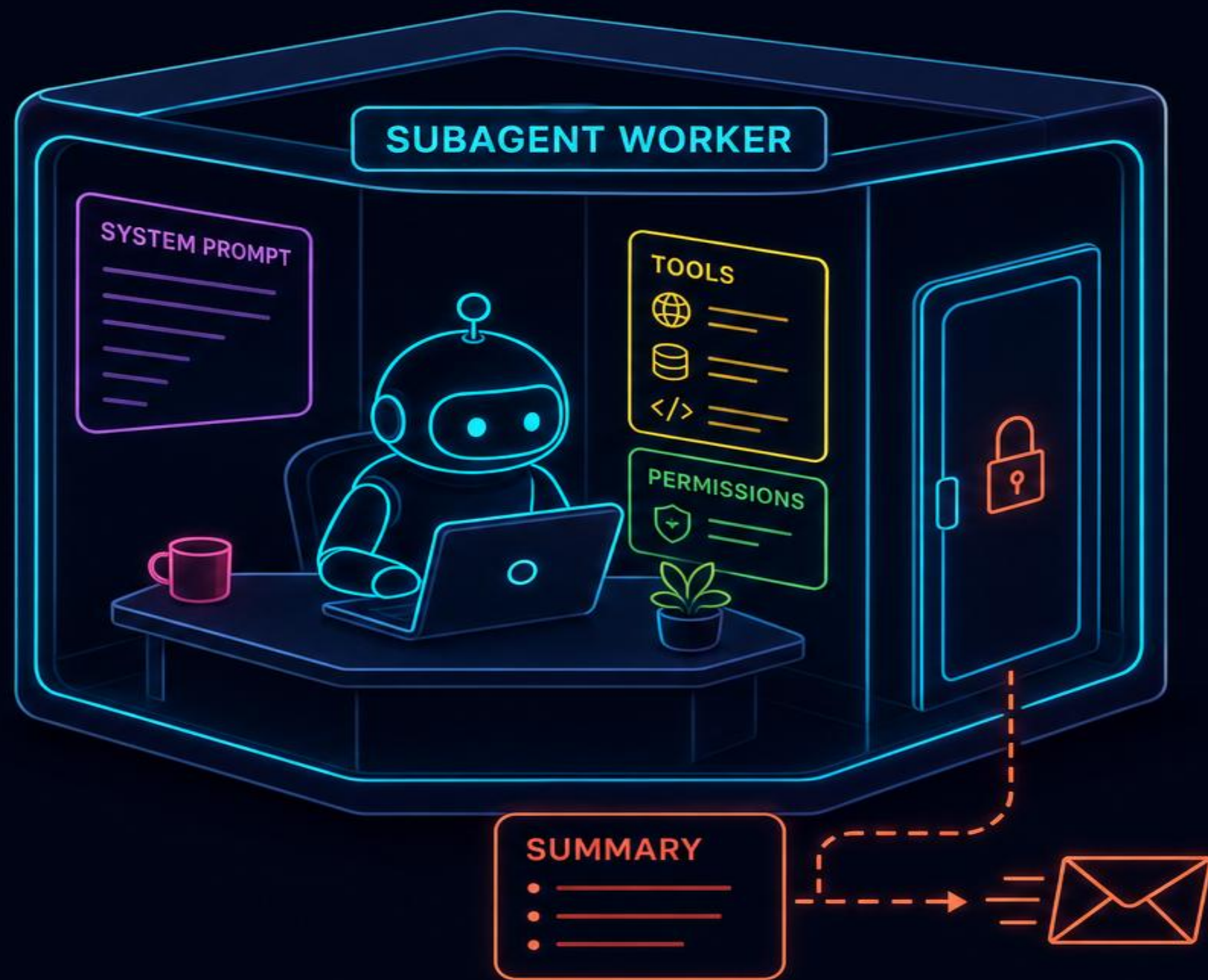
It doesn't see
your chat history or
the files you've
already read



It runs its own
system prompt,
tools, and
permissions



When it finishes,
only a summary
comes back to you



How You Define a Subagent

A subagent is a Markdown file with YAML frontmatter, and the body is its system prompt.



research-planner.md

```
1 ---
2 name: research-planner
3 description: Breaks complex topics into
4   research plans with queries and sources
5 model: claude-opus-5
6 tools: WebSearch, WebFetch, Read
7 permissionMode: plan
8 ---
9 # Research Planner
10 You are a senior research planner.
11 Given a topic, produce a structured research
12   plan with key questions, search queries,
13   and high-quality sources.
14 Ask clarifying questions if the topic
15   is ambiguous. Prefer recent, authoritative
16   sources and cite URLs.
```



Only name and description are required fields

A subagent must have a name and description to be valid.



Tools, model, and permissions are all optional

Specify them to customize behavior, or omit to rely on defaults.



Leave tools out, and it inherits the full set

If you don't list tools, the subagent inherits all available tools.

Subagent vs. Main Agent



Main agent

Holds the full conversation. It sees everything you've said, every file read, and every skill invoked so far in the current session.



Subagent

Starts fresh with none of that. It works the task in isolation, then hands back a summary, keeping the main context clean and bounded.

Three Ways to Invoke



Natural language

Ask Claude to use a named subagent, and it decides whether to delegate based on the description.



At-mention

Mention the agent by name to guarantee that specific subagent runs for one task.



Agent flag

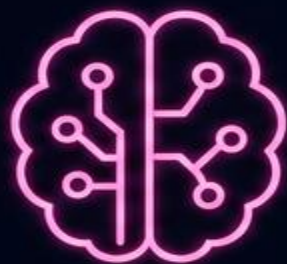
Start with the agent flag, and the whole session runs as that subagent from the start.

Agent Memory Is Not CLAUDE.md

People lump agent memory in with CLAUDE.md, but they're different things.



CLAUDE.md is instructions you write for Claude to follow



Agent memory is durable state the agent writes itself



Memory survives compaction and outlives a single session



agent_memory.md

Self-updated by the agent



Goals

- Ship v1.2 by May 28
- Improve user onboarding



Insights

- Users get stuck at API key step
- Error rate drops after retry logic



Tools & Preferences

- Prefer web_search over docs
- Use get_weather for forecasts



Facts & Decisions

- Timezone: America/Los_Angeles
- Chose Supabase over Firebase



History (compacted)

- 2025-05-20: Added retry logic
- 2025-05-21: Switched auth flow

Author Rules



Write durable facts, not ephemeral thoughts.



Keep it brief, accurate, and actionable.



Never store secrets or sensitive data.



Update after meaningful outcomes.



Compact old history periodically.

Two Memory Systems



Main-agent auto memory

Claude writes notes to itself from your corrections. It's on by default and lives in a per-project folder on your machine, not synced elsewhere.

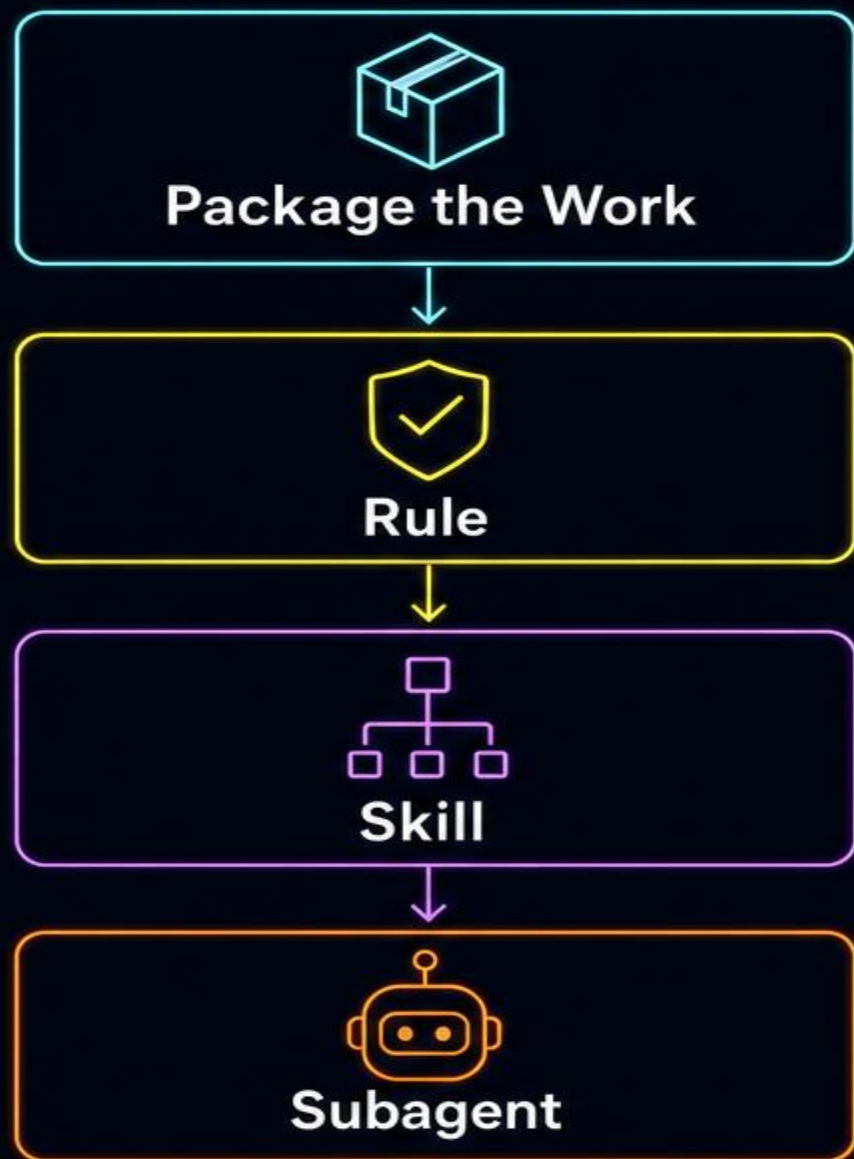


Subagent memory

A subagent's own store, turned on by a frontmatter field. It lets that agent carry knowledge across sessions on its own, deliberately.

Choosing the Right Mechanism

When you're deciding how to package something, three questions point the way.



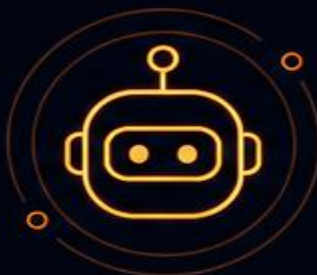
Always-apply policy? Reach for a rule

If the decision should be consistent, audit-able, and applied the same way every time, encode it as a rule so it's always on.



On-demand workflow? Package it as a skill

If users or systems call for it sometimes to get a job done, package the end-to-end workflow as a skill they can invoke.



Scope-isolated work? Delegate to a subagent

If the work needs its own context, tools, or state and should not leak into others, hand it off to a subagent to run in isolation.

Match the mechanism to the job, and your setup stays clean.

Delegation and Memory



A subagent runs in an isolated context, returning a summary



Defining one needs only a name and description



Invoke it by asking, by name, or by flag



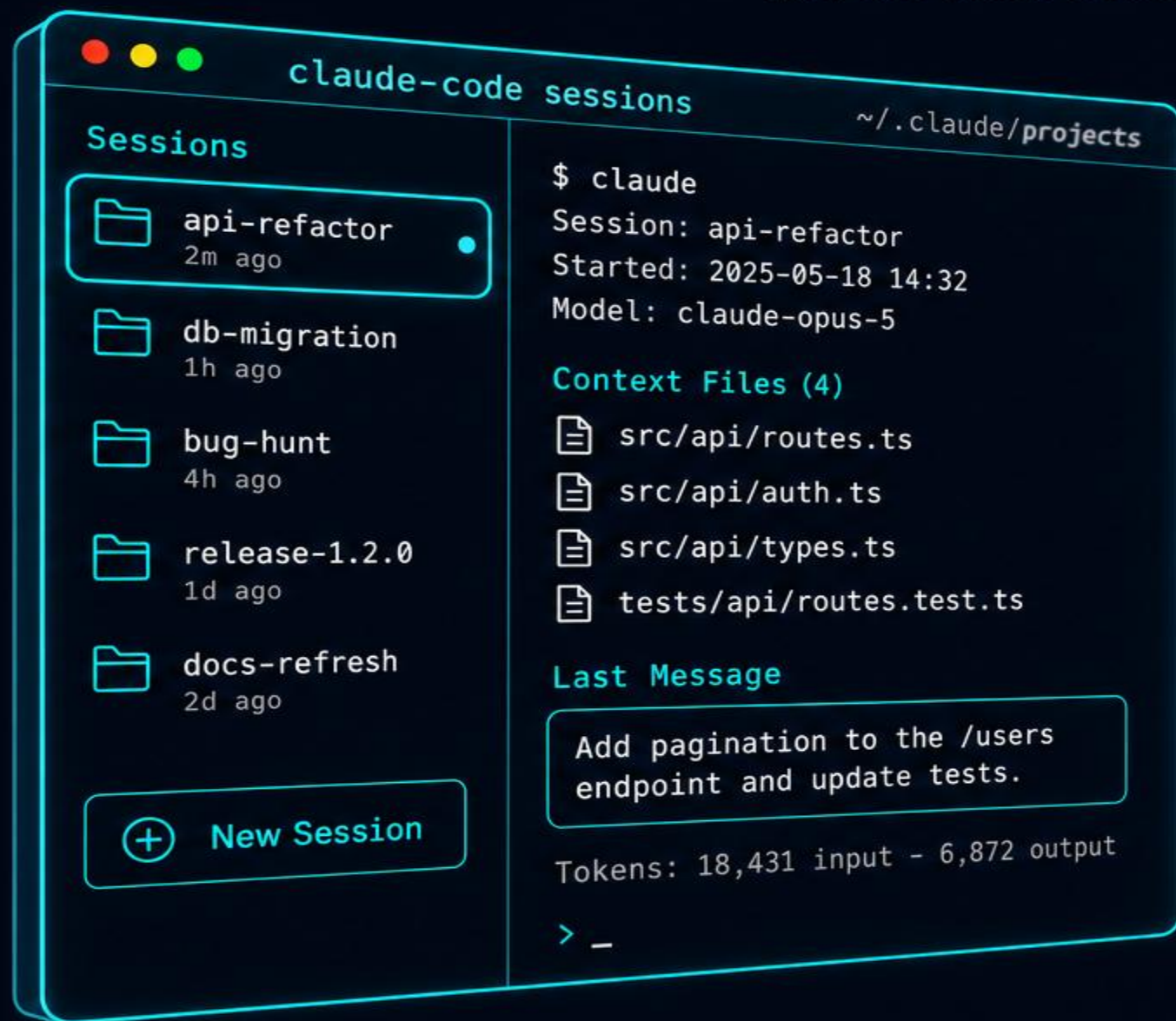
Agent memory is durable state, distinct from CLAUDE.md



Two systems exist, auto memory and subagent memory

Sessions and Slash Commands

Every conversation in Claude Code is a session saved on disk,
and the slash-command menu is how you steer it.



Three Terms to Start



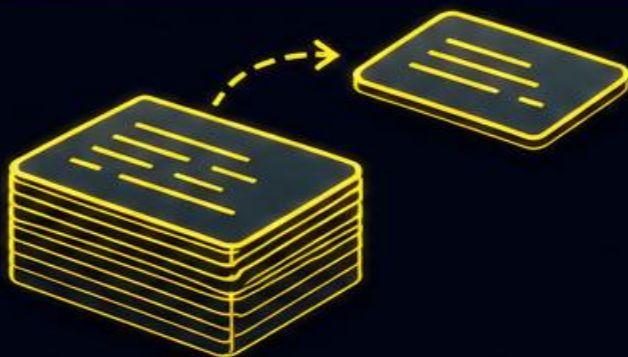
Session

A single conversation tied to the project directory you launched it in, saved to disk as you go so you can return to it.



Slash command

A shortcut you type with a leading slash to run an action, from clearing context to invoking a workflow your team wrote.



Compaction

Replacing older history with a shorter summary when the window fills, freeing space while keeping your recent thread alive.

Where a Session Lives

A session isn't just the text scrolling on your screen; it's a record on disk you can return to.



Each session is tied to the project directory you launched it in



Transcripts write continuously, so closing the terminal loses nothing

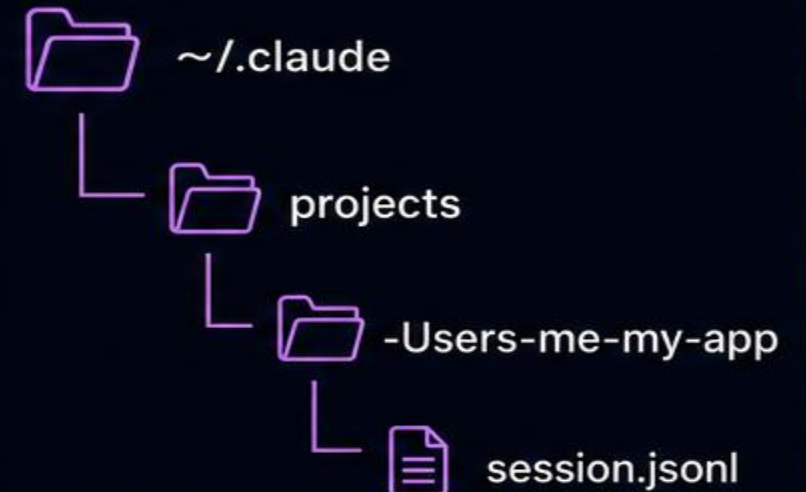


Old sessions clear after a retention window you can configure

PROJECT DIRECTORY



SESSION TRANSCRIPTS



Treat the raw transcript as internal, and reopen your work through the commands.

Three Ways to Reopen Work



Continue

Launching with the continue flag reopens the most recent session in your current directory, right where you left off.



Resume

The resume flag opens a picker of your past sessions, or jumps straight to one you name.



In-session resume

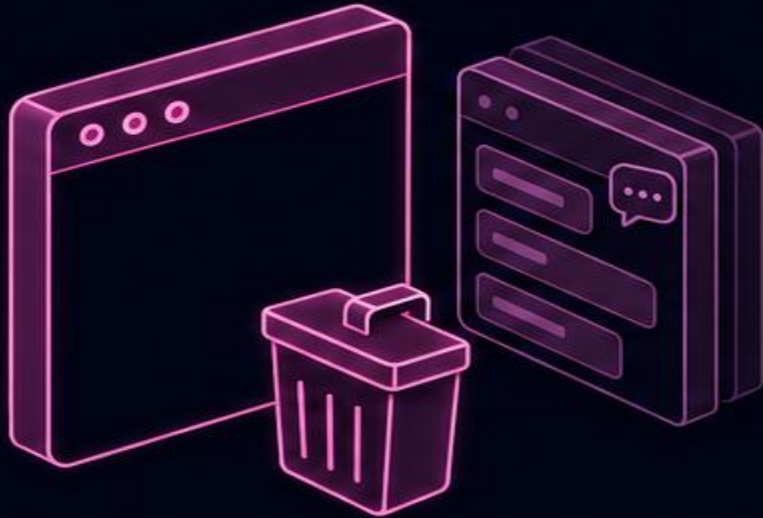
Resuming from inside a live session switches you to a different past conversation without quitting first.

Compact vs. Clear



Compacting

Replaces older turns with a summary when the window fills. Your recent decisions and current task survive in condensed form, so you keep working.



Clearing

Empties the window and starts a new conversation. Nothing from before carries forward, yet the previous one stays saved and resumable.

No Memory Between Sessions

Claude Code doesn't remember your last session on its own; a fresh one starts blank.



Four Built-In Command Groups



Session and context

Commands that reopen, reset, or summarize a conversation, like resume, clear, and compact.



Model and config

Commands that switch the model, effort level, or a setting without leaving your session.



Setup and memory

Commands that scaffold a repo or manage instructions, like init and memory.



Quality and workflow

Bundled skills that run a whole procedure for you, from code review to planning.

Three Facts About Custom Commands



Filename sets the name

The file or directory name becomes what you type after the slash, not any field written inside the file.



Location sets the scope

In the project folder a command is shared through version control; in your home folder it stays personal to you.



Commands are skills

A custom command and a skill are one system now, so a single file gives you a reusable, shareable action.

A Session Hygiene Loop

Good session habits are a short loop you run deliberately, not a one-time cleanup.



Prune stale tool output that's already done its job



Compact when the window fills but the thread still matters



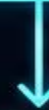
Clear or start fresh when the context turns into baggage



Resume a saved session when you need that history back

①

Prune



②

Compact



③

Clear



④

Resume



Steering Your Sessions



Sessions save to disk and reopen with continue or resume



Compact summarizes, clear empties, and neither deletes your history



Continuity never comes free, it comes from CLAUDE.md, memory, and resume



Built-ins ship with the CLI, custom commands you write

Headless Mode, Config, and Repo Init

Running Claude Code by hand is one mode. Wiring it into scripts, pipelines, and shared repos is another.

headless job running unattended

```
$ claude -p --settings ~/.claude/settings.json \  
  "Implement login with email magic link"
```

```
[10:23:11] • Starting headless session  
[10:23:12] • Loaded settings: ~/.claude/settings.json  
[10:23:13] • Model: claude-opus-5  
[10:23:13] • Tools: Read, Write, Edit, Bash  
[10:23:14] • Planning  
[10:23:18] • Step 1/5 Read src/auth/routes.ts  
[10:23:22] • Step 2/5 Write src/auth/magic-link.ts  
[10:23:29] • Step 3/5 Edit src/auth/routes.ts  
[10:23:34] • Step 4/5 Bash npm test -- src/auth  
[10:23:46] • Tests passed (18)  
[10:23:48] • Step 5/5 Commit changes  
[10:23:51] • Completed in 40s
```

Summary:

- Files changed: 3
- Tests: 18 passed
- Commit: 9f3a2bd feat(auth): magic link login
- Branch: feature/magic-link

```
$ █
```



team/app
Git repository



Runner

```
claude -p --settings  
~/.claude/settings.json  
Tools: Read, Write, Bash
```

Three Terms to Anchor



Headless Mode

Running Claude Code non-interactively with the `-p` or `--print` flag. It takes a prompt, runs to completion, prints to stdout, and exits without waiting for input.



settings.json

The JSON config file holding permissions, hooks, and environment. It layers across user, project, and a personal, git-ignored local file.



CLAUDE.md

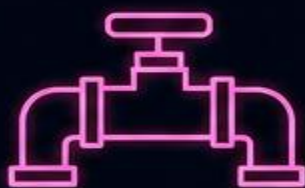
Markdown instructions loaded into context every session. Layers are concatenated in scope order, project read last, so keep them consistent.

Why the Print Flag Matters

Automation needs a run that finishes on its own. That's what the print flag gives you.



-p runs the task and prints the result to stdout

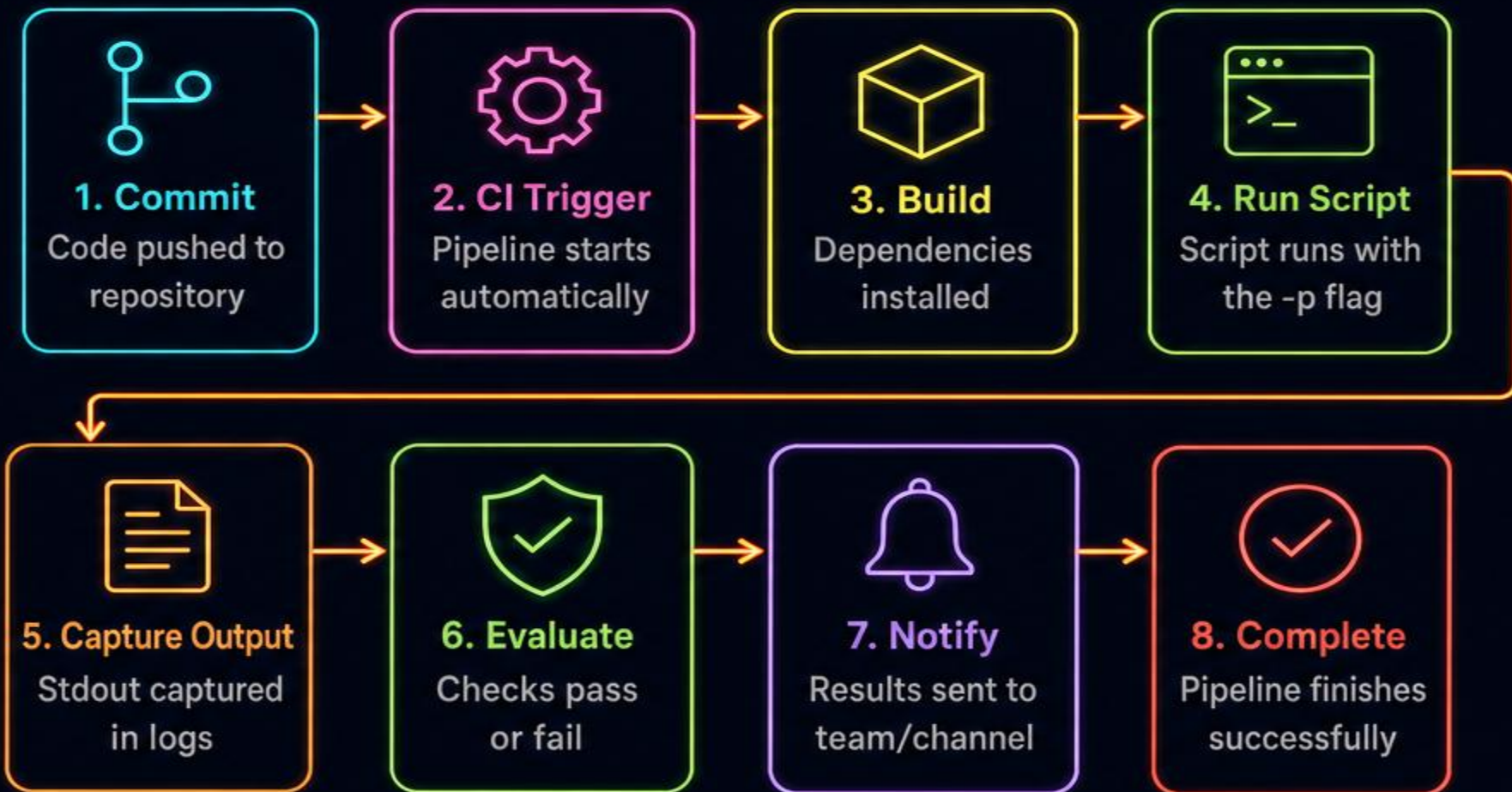


It reads stdin, so you can pipe data in like any Unix tool



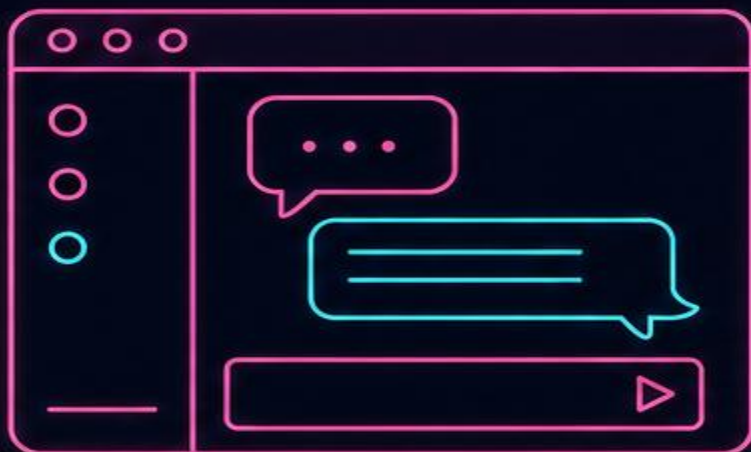
Without it, Claude Code opens an interactive session and waits

Unattended script running in a CI pipeline



A pipeline job that waits for input nobody types just hangs.

Interactive vs. Headless



Interactive

You launch a session, converse turn by turn, and approve actions as they come. Great for exploration, wrong for a script that has to run unattended.

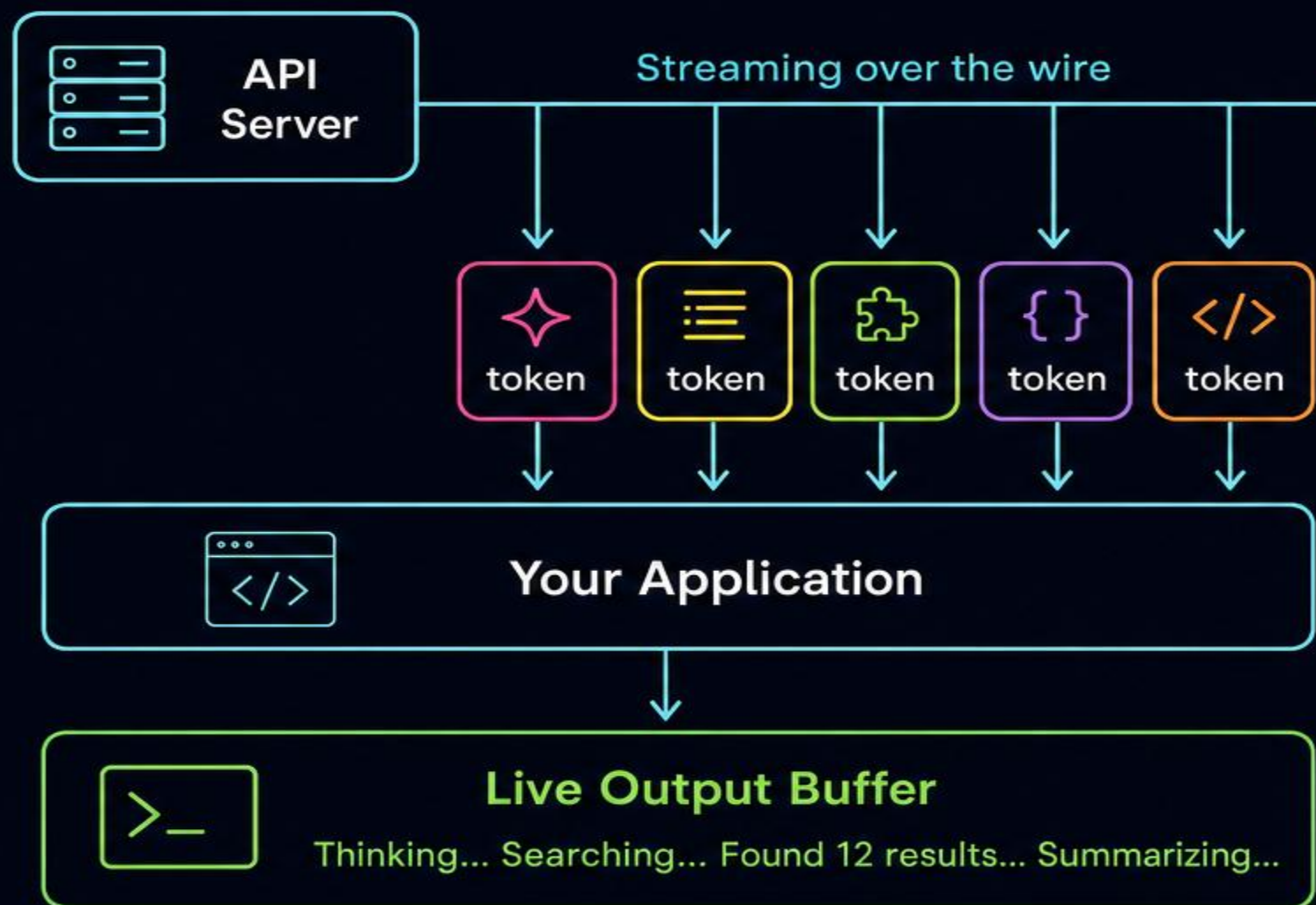


Headless

You pass one prompt with the print flag, Claude runs to completion, prints, and exits. This is the mode for CI/CD, cron jobs, and shell scripts.

Streaming Output for Long Runs

For programmatic use, you choose how results come back.



Plain text is the default, one block when the run ends



Structured JSON returns the result plus session and cost data



stream-json emits events line by line as work happens

In the SDK, streaming input is the recommended mode, and stream-json feeds progress as it happens.

The Approval-Mode Spectrum



Default (Manual)

Claude asks before every edit and tool call.
Full oversight, but it stops constantly.



Accept-edits

Applies file edits and common filesystem commands without a prompt. Shell commands beyond that set still ask.



Plan

Proposes the actions first and waits for your go-ahead, so review comes before acting.



dontAsk

The documented mode for unattended or CI runs.
Only pre-approved tools run, and the session never waits.

Where the Guardrails Live

Skip the approval prompt, and your safety net moves into configuration.



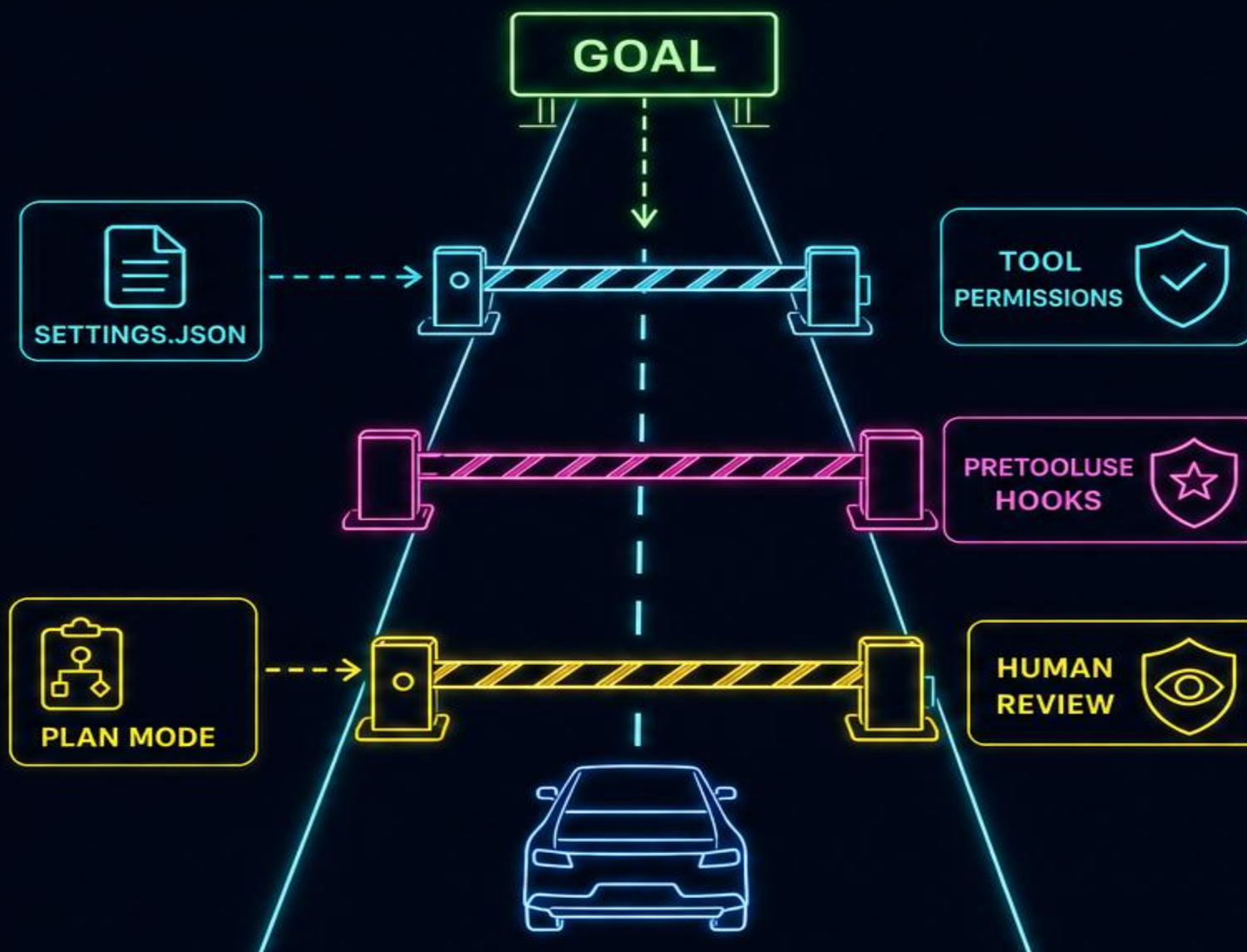
permissions in settings.json
decide what tools can run at all



PreToolUse hooks can block a dangerous action before it fires



Plan mode stays the answer when a human review is required



Automation speed is safe only when the guardrails are wired in first.

The CLAUDE.md Hierarchy



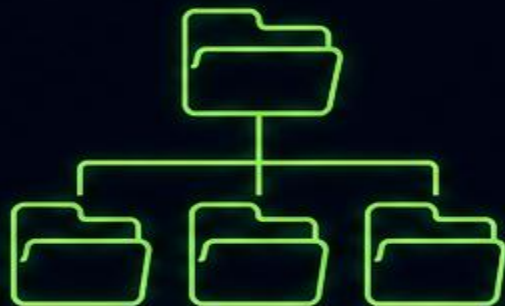
User level

~/.claude/CLAUDE.md applies to all your projects. It loads first as the broadest, lowest-priority layer.



Project level

The repo-root CLAUDE.md, shared with your team in version control. It loads after the user file, but nothing enforces precedence, so design conflicts out.

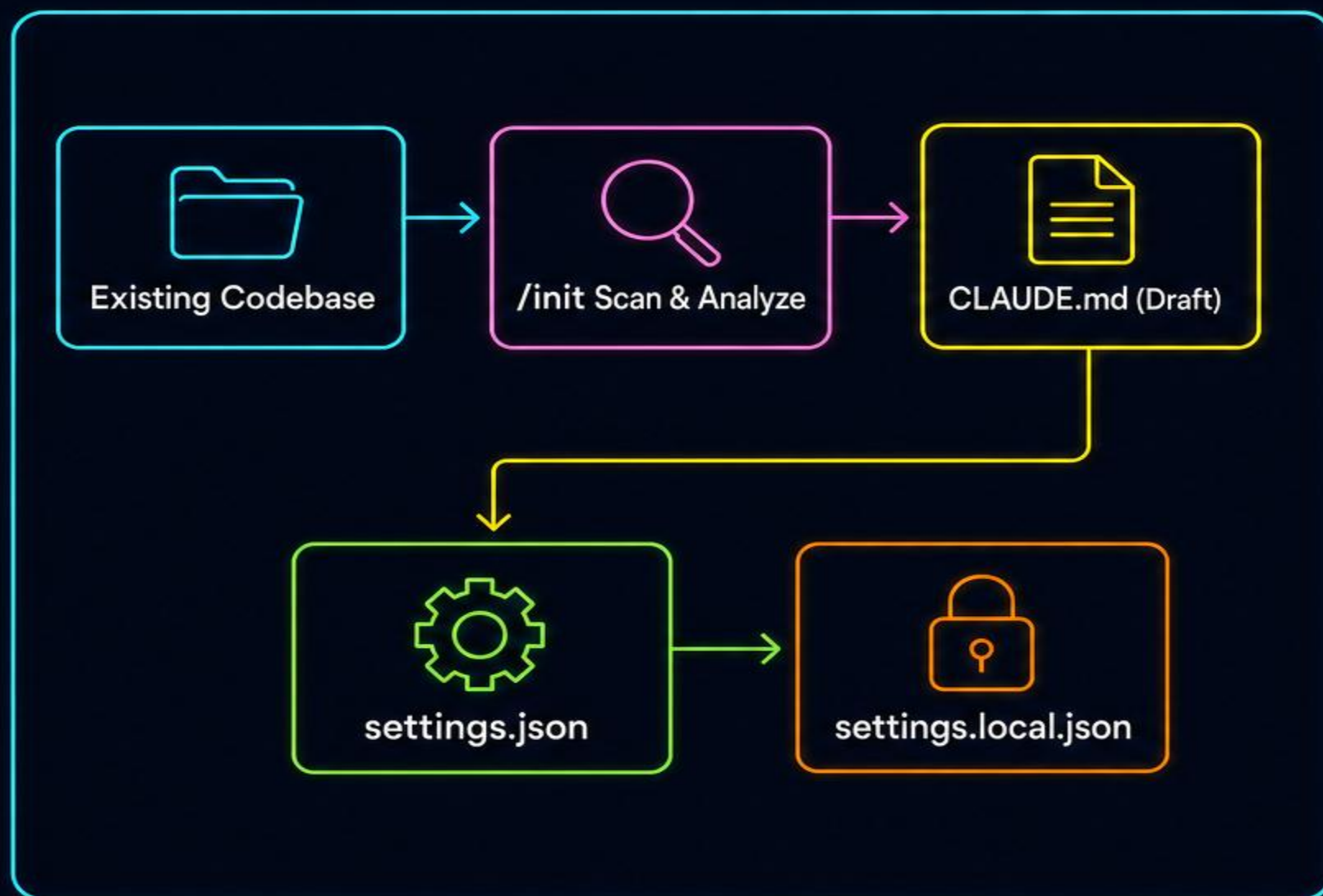


Subdirectory

A CLAUDE.md deeper in the tree scopes rules to that folder, loading when Claude works with files there.

Init a Repo, Layer Your Settings

A new repo gets configured cleanly with init and settings.json.



/init scans the repo and drafts a starter CLAUDE.md to review



Treat that draft as a first pass, never a finished config



settings.json holds permissions, hooks, and env for the team



settings.local.json is personal and stays out of version control

Wiring Claude Code Into Your Stack



Use the print flag for any non-interactive, unattended run



Reach for stream-json when you need incremental output



Move guardrails to permissions and hooks when approval is off



Layer CLAUDE.md by scope, and remember it costs context



Draft with /init, then configure permissions in settings.json